CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering, Mini-Project

Ashok Reddy (IMT2023083) · 30 October 2026

GitHub: https://github.com/ashokCh-dev/calculator-devops
Docker Hub: https://hub.docker.com/r/ashokch11/calculator-devops

1. Overview

A Spring Boot 3.5 (Java 21) REST API with GET `/api/v1/{add,subtract,multiply,divide}?a=&b=` and `/actuator/health`. Invalid input and division by zero return HTTP 400 with a JSON error. Every push to GitHub is built, tested, containerised, pushed to Docker Hub and deployed automatically, and Jenkins emails the result.

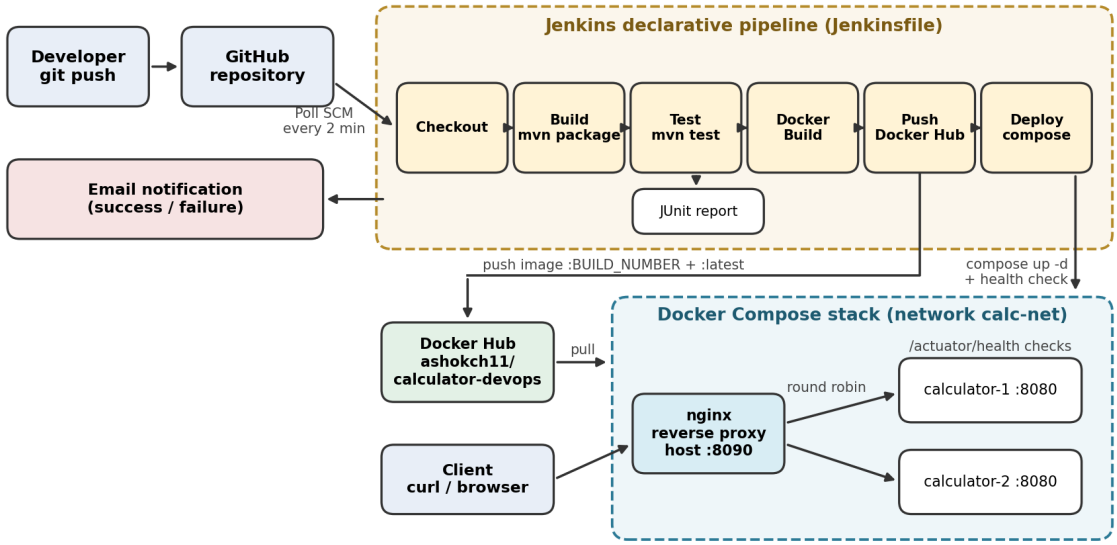


Figure 1: CI/CD pipeline architecture

2. Pipeline Stages (Jenkinsfile)

Stage	What it does
Trigger	`pollSCM('H/2 * * * *')`: starts a build when GitHub has a new commit.
Checkout	`checkout scm` gets the commit that triggered the build.
Build	`./mvnw clean package -DskipTests` (Maven wrapper, so no Maven install is needed).
Test	`./mvnw test`: 28 JUnit 5 + MockMvc tests; results published with `junit`; a failure fails the build.
Docker Build	Builds the image tagged `:<BUILD_NUMBER>` and `:latest`.
Push	Logs in to Docker Hub with the Jenkins credential `dockerhub-creds` and pushes both tags.
Deploy	`docker compose up -d`, then waits for `/actuator/health` = UP through Nginx and runs a smoke test.
Post	`emailext` sends a success or failure email; dangling images are pruned.

Dockerfile: multi-stage (Maven/JDK build, then `eclipse-temurin:21-jre-alpine` runtime), dependency layer caching, non-root user, `EXPOSE 8080`, `HEALTHCHECK`, `.dockerignore`. **Compose:** two API replicas behind an Nginx reverse proxy (round robin, port 8090) on network `calc-net`, with health checks, `restart: unless-stopped` and environment variables. Re-runs are safe: `up -d` only recreates changed containers. **Git:** progressive commits, feature branches `feature/docker-compose` and `feature/jenkins-pipeline` merged through pull requests, Java `.gitignore`, no secrets in the repo.

3. Challenges Faced

1. **No webhook:** Jenkins runs on localhost, so GitHub cannot reach it. I used Poll SCM every 2 minutes.
2. **Port clash:** Jenkins uses 8080, so only Nginx is published, on port 8090.
3. **Load balancing:** every request hit one replica because each Nginx worker kept its own round-robin state. A shared `zone` in the upstream block fixed it.
4. **Maven setup:** the `jenkins` user had no Maven. The committed Maven wrapper fixed this.
5. **Health checks:** the Alpine image has no `curl`, so the health checks use `wget`.
6. **Email:** the notification address did not match the Gmail account used for SMTP; I fixed the address in the Jenkinsfile.

4. Results

All stages pass on every push (builds #3 and #4 were started automatically by an SCM change). The image is on Docker Hub and the stack is healthy.

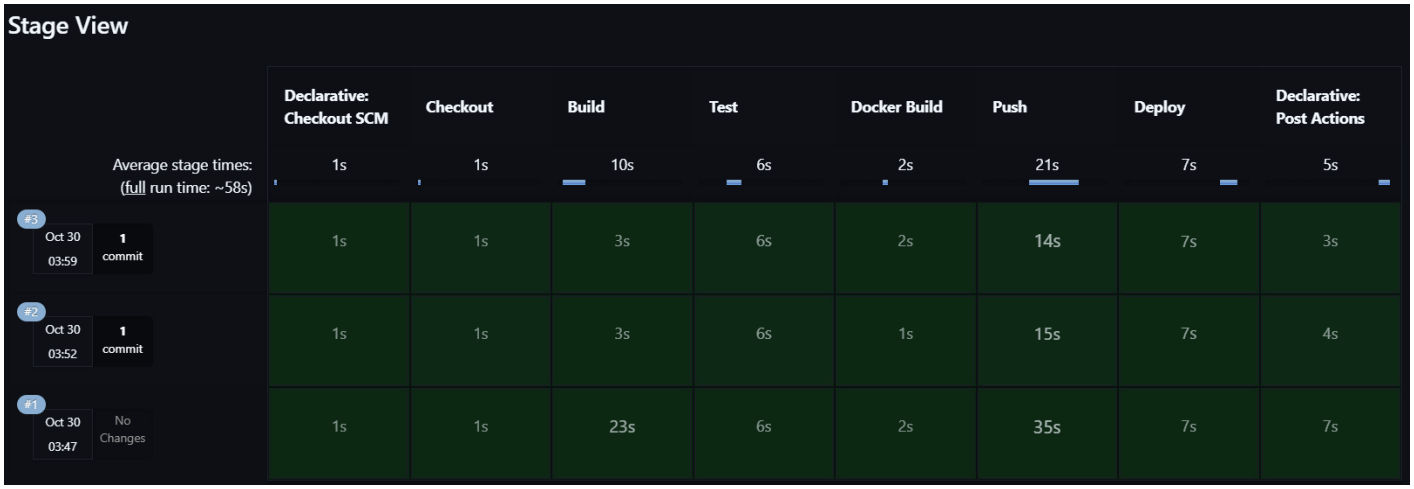


Figure 2: Jenkins stage view: all stages green

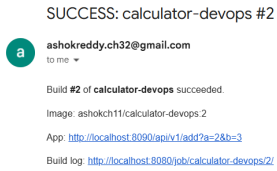


Figure 3: Build status email

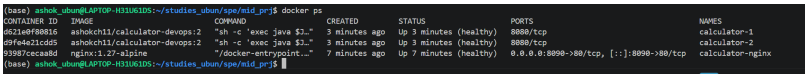


Figure 4: `docker ps`: replicas and proxy healthy

```
(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl "http://localhost:8090/api/v1/add?a=2&b=3"; echo
{"operation":"add","a":2.0,"b":3.0,"result":5.0}
(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl "http://localhost:8090/api/v1/divide?a=10&b=4"; echo
{"operation":"divide","a":10.0,"b":4.0,"result":2.5}
(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl "http://localhost:8090/api/v1/divide?a=1&b=0"; echo
{"timestamp":"2026-10-29T22:27:10.757695909Z","status":400,"error":"Bad Request","message":"Division by zero is not allowed","path":"/api/v1/divide"}
(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl "http://localhost:8090/api/v1/add?a=abc&b=1"; echo
{"timestamp":"2026-10-29T22:27:21.242290543Z","status":400,"error":"Bad Request","message":"Parameter 'a' must be a number, got 'abc'","path":"/api/v1/add"}
(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl -i "http://localhost:8090/api/v1/multiply?a=6&b=7"
HTTP/1.1 200
Server: nginx/1.27.5
Date: Thu, 29 Oct 2026 22:27:27 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
X-Upstream: 172.20.0.3:8080

{"operation":"multiply","a":6.0,"b":7.0,"result":42.0}(base) ashok_ubun@LAPTOP-H31U61DS:~/studies_ubun/spe/mid_prj$ curl "http://localhost:8090/actuator/health"; echo
{"status":"UP","groups":["liveness","readiness"],"components":{"diskSpace":{"status":"UP","details":{"total":1081101176832,"free":893433016320,"threshold":10485760,"path":"/app/.",
"exists":true}},"livenessState":{"status":"UP"},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"ssl":{"status":"UP","details":{"validChains":[],"invalidChains":[]}}}}
```

Figure 5: Sample API responses through Nginx (port 8090)